

1) Predict the output

```
#include <stdio.h>
int main() {
    int x = 5, y = 3;
    int result = x & y << 1;
    printf("%d\n", result);
    return 0;
}
```

- a) 4
- b) 1
- c) 6
- d) 0

2) Which of the following expressions evaluates to true if and only if a positive integer n is a power of 2?

- a) $(n \& (n - 1)) == 0$
- b) $(n \mid (n + 1)) == 0$
- c) $(n \wedge (n - 1)) == 0$
- d) $(n \& \sim n) == 1$

3) Find the bug

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;
    a ^= b ^= a ^= b;
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

- a) a = 20, b = 10
- b) a = 0, b = 10
- c) Undefined behavior due to sequence point violations
- d) Compilation error

4) Output?

```
#include <stdio.h>

int main() {
    unsigned int n = 43; // Binary: 101011
    int count = 0;
    while (n) {
        n = n & (n - 1);
        count++;
    }
}
```

```

    }
    printf("%d\n", count);
    return 0;
}

```

- a) 3
- b) 4
- c) 5
- d) 6

5) Output?

```
#include <stdio.h>
```

```

int main() {
    signed char c = -128;
    unsigned char uc = 128;
    c = c >> 1;
    uc = uc >> 1;
    printf("%d, %d\n", c, uc);
    return 0;
}

```

- a) -64, 64
- b) -64, 192
- c) Implementation defined behavior for signed, 64 for unsigned
- d) Compilation error

6) Guess the output

```
#include <stdio.h>
```

```

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *ptr = arr;
    printf("%d ", *(ptr + 3));
    printf("%d\n", ptr[2]);
    return 0;
}

```

- a) 30 30
- b) 40 30
- c) 40 40
- d) 30 40

7) Which statement correctly declares a pointer to an array of 5 integers?

- a) int *ptr[5];
- b) int (*ptr)[5];
- c) int *(*ptr) (5);

d) `int ptr[5]*;`

8) `#include <stdio.h>`

```
int* get_local() {  
    int x = 42;  
    return &x;  
}
```

```
int main() {  
    int *p = get_local();  
    printf("%d\n", *p);  
    return 0;  
}
```

- a) Prints 42
- b) Compilation warning: function returns address of local variable
- c) Undefined behavior
- d) Prints garbage value 0

9) `#include <stdio.h>`

```
int add(int a, int b) { return a + b; }  
int sub(int a, int b) { return a - b; }
```

```
int main() {  
    int (*fp[2])(int, int) = {add, sub};  
    printf("%d\n", fp[1](10, 5));  
    return 0;  
}
```

- a) 15
- b) 5
- c) Compilation error
- d) Segmentation fault

10) `#include <stdio.h>`

```
void test(int arr[]) {  
    int x = sizeof(arr);  
    int y = sizeof(int *);  
    printf("%d %d\n", x, y);  
}
```

```
int main() {
```

```

int arr[10];
test(arr);
return 0;
}

```

- a) 40 40 (on 64-bit systems, 8 8)
- b) 40 8 (on 64-bit systems)
- c) 8 8 (on 64-bit systems)
- d) Compilation error

11) #include <stdio.h>

```

struct Data {
    char c;
    int i;
    short s;
};

int main() {
    printf("%zu\n", sizeof(struct Data));
    return 0;
}

```

- a) 7
- b) 8
- c) 12
- d) 16

12) #include <stdio.h>

```

union Test {
    int i;
    char arr[4];
};

int main() {
    union Test t;
    t.i = 0x01020304;
    printf("%d\n", t.arr[0]);
    return 0;
}

```

- a) Prints 1 on big-endian, 4 on little-endian
- b) Prints 4 on big-endian, 1 on little-endian
- c) Always prints 1
- d) Undefined behavior

13) #include <stdio.h>

```
typedef struct {  
    int x;  
    int y;  
} Point;
```

```
int main() {  
    Point p = {10, 20};  
    Point *ptr = &p;  
    printf("%d\n", ptr->x + (*ptr).y);  
    return 0;  
}
```

- a) 10
- b) 20
- c) 30
- d) Compilation error

14) #include <stdio.h>

#include <stdlib.h>

```
typedef struct {  
    int *ptr;  
} Holder;
```

```
int main() {  
    Holder h1;  
    h1.ptr = (int *)malloc(sizeof(int));  
    *h1.ptr = 100;  
  
    Holder h2 = h1;  
    *h2.ptr = 200;  
  
    printf("%d\n", *h1.ptr);  
    free(h1.ptr);  
    return 0;  
}
```

- a) 100
- b) 200
- c) Runtime double free error
- d) Memory leak

15) #include <stdlib.h>

```
void process() {  
    int *ptr = (int *)malloc(100 * sizeof(int));  
    if (ptr == NULL) {  
        return;  
    }  
    // computations  
    if (some_condition()) {  
        return; // Bug location  
    }  
    free(ptr);  
}
```

- a) Missing free(ptr) before conditional return
- b) Invalid cast of malloc return value
- c) Allocation size overflow
- d) No memory leak present

16) #include <stdlib.h>

```
int main() {  
    int *p = (int *)malloc(sizeof(int));  
    *p = 5;  
    free(p);  
    // ... code ...  
    free(p);  
    return 0;  
}
```

- a) Memory leak
- b) Undefined heap corruption
- c) Compilation error
- d) Dangling pointer read (safe write)

17) #include <stdlib.h>

```
int main() {  
    int *ptr = (int *)malloc(10 * sizeof(int));  
    ptr = (int *)realloc(ptr, 5 * sizeof(int));  
    // Is this code safe?  
    return 0;  
}
```

- a) Perfectly safe; shrinks allocation and keeps pointer valid

- b) Causes memory leak if realloc fails and returns NULL
- c) Causes buffer overflow
- d) Compilation error

18) `def add_item(item, target_list=[]):`
 `target_list.append(item)`
 `return target_list`

`print(add_item(1))`
`print(add_item(2))`

- a) [1] followed by [2]
- b) [1] followed by [1, 2]
- c) [1, 2] followed by [1, 2]
- d) TypeError

19) `s = "PythonOA"`
`print(s[10:15])`
`print(s[-20:3])`

- a) IndexError on both lines
- b) "" and "Pyt"
- c) "PythonOA" and ""
- d) None and "Pyt"

20) `a = 256`
`b = 256`
`c = 257`
`d = 257`
`print(a is b, c is d)`

- a) True True
- b) True False
- c) False False
- d) False True

21) `matrix = [[0] * 3] * 3`
`matrix[0][0] = 1`
`print(matrix)`

- a) [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
- b) [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
- c) [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
- d) TypeError

22) `gen = (x ** 2 for x in range(3))`
`print(sum(gen))`

```
print(list(gen))
```

- a) 5 and [0, 1, 4]
- b) 5 and []
- c) TypeError: 'generator' object is not subscriptable
- d) 0 and [0, 1, 4]

```
23) funcs = [lambda x: x * i for i in range(3)]
```

```
print([f(2) for f in funcs])
```

- a) [0, 2, 4]
- b) [4, 4, 4]
- c) [0, 0, 0]
- d) [2, 2, 2]

```
24) def test_func():
```

```
    try:
```

```
        return 1
```

```
    except:
```

```
        return 2
```

```
    finally:
```

```
        return 3
```

```
print(test_func())
```

- a) 1
- b) 2
- c) 3
- d) Syntax Error

```
25) import copy
```

```
original = [[1, 2], [3, 4]]
```

```
shallow = copy.copy(original)
```

```
deep = copy.deepcopy(original)
```

```
original[0][0] = 99
```

```
print(shallow[0][0], deep[0][0])
```

- a) 1, 1
- b) 99, 1
- c) 99, 99
- d) Attribute Error

```
26) names = ['Alice', 'Bob']
```

```
scores = [85, 90]
```

```
for idx, (name, score) in enumerate(zip(names, scores)):
```



```
if idx == 1:  
    print(name, score)
```

- a) Alice 85
- b) Bob 90
- c) Alice 90
- d) Index Error

27) count = 10

```
def increment():  
    count += 1  
    return count
```

```
print(increment())
```

- a) 11
- b) 10
- c) UnboundLocalError: local variable 'count' referenced before assignment
- d) NameError

28) A shopkeeper marks up an article by 40% above its cost price and gives a successive discount of 20%. What is his overall profit percentage?

- a) 10%
- b) 12%
- c) 15%
- d) 20%

29) A can complete a piece of work in 12 days, and B can complete it in 18 days. They work together for 4 days, after which A leaves. In how many days will B complete the remaining work?

- a) 6 days
- b) 8 days
- c) 10 days
- d) 12 days

30) A train running at a speed of 90 km/h crosses a 300-meter-long platform in 20 seconds. What is the length of the train?

- a) 150 meters
- b) 200 meters
- c) 250 meters
- d) 300 meters

31) The difference between compound interest (compounded annually) and simple interest on a certain sum of money for 2 years at 10% per annum is \$40. Find the principal sum.

- a) \$3,500
- b) \$4,000
- c) \$4,500
- d) \$5,000

32) In what ratio must water be mixed with milk costing \$32 per litre so that the mixture can be sold at \$28 per litre, making a profit of 25%?

- a) 1:2
- b) 2:3
- c) 1:3
- d) 3:7

33) A bag contains 4 red, 3 green, and 5 blue balls. Three balls are drawn at random. What is the probability that all three balls are of different colors?

- a) $\frac{3}{11}$
- b) $\frac{12}{33}$
- c) $\frac{3}{22}$
- d) $\frac{16}{33}$

34) In how many ways can the letters of the word "ENGINEERING" be arranged such that all the E's are never together?

- a) 277,200
- b) 275,180
- c) 262,080
- d) 265,400

35) What is the unit digit of the expression: $7^{95} \times 3^{58} \times 9^{41}$

- a) 1
- b) 3
- c) 7
- d) 9

36) The LCM of two numbers is 480 and their ratio is 3:4. What is the sum of the two numbers?

- a) 280
- b) 320
- c) 360
- d) 400

37) The average weight of 30 students in a class is 45 kg. If the weight of the teacher is included, the average weight increases by 1 kg. What is the weight of the teacher?

- a) 72 kg
- b) 75 kg
- c) 76 kg
- d) 80 kg

38) Two pipes A and B can fill a tank in 15 hours and 20 hours respectively, while a third pipe C can empty the full tank in 12 hours. If all three pipes are opened simultaneously, in how much time will the empty tank be filled?

- a) 10 hours
- b) 15 hours
- c) 25 hours
- d) 30 hours

39) At what time between 4:00 and 5:00 o'clock will the minute hand and hour hand of a clock coincide?

- a) 4 hrs 21 $\frac{9}{11}$ mins
- b) 4 hrs 20 $\frac{4}{11}$ mins
- c) 4 hrs 22 $\frac{6}{11}$ mins
- d) 4 hrs 18 $\frac{2}{11}$ mins

ANSWERS:

- 1) A (4)
Bitwise shift op has higher precedence than bitwise AND
 $y \ll 1 \Rightarrow 6$
 $5 \& 6 \Rightarrow 4$
- 2) A ($n \& (n-1) == 0$)
Powers of 2 in binary has exactly one set bit (0100, 1000)
Subtracting 1 from power 2 flips all bits from set bit
 $4 - 1 = 3 \Rightarrow 0011$
 $8 - 1 = 7 \Rightarrow 0111$
Bitwise And between n and n-1 results in 0
- 3) C (Undefined behavior due to seq point violations)
The program attempts to swap 2 variables
But order of sub expressions and side effects in a single expression without a Sequence point ($\&\& \parallel , ;$) is undefined
- 4) B (4)
Iter 1 : $n = 43 \mid n = 42 \mid \text{count } 1$
Iter 2 : $n = 42 \mid n = 40 \mid \text{count } 2$
Iter 3 : $n = 40 \mid n = 32 \mid \text{count } 3$
Iter 4 : $n = 32 \mid n = 0 \mid \text{count } 4$
out
- 5) C (Implementation defined behavior for signed and unsigned)
Unsigned char uc = 128 (10000000) right shift (01000000) which is 64
signed char c = -128 (10000000) right shifting a neg signed int is implementation defined
not guaranteed across all architecture
- 6) B (40 30)
arr is initialized with values, ptr points to arr[0]
 $*(ptr + 3)$ moves and points to arr[3] $\Rightarrow 40$
 $ptr[2] == *(ptr+2) \Rightarrow 30$
- 7) B $\text{int } (*ptr) [5];$
A pointer to 5 integers
[] and () binds before *
 $\text{int } *ptr[5]$ defines 5 pointers to integers
- 8) C undefined behavior
`get_local()` defines a local variable x, when returned the stack frame is destroyed
Derefering p in printf leads to undefined behavior or segmentation fault

9) B (5)

Array of size 2 function pointers

fp[1] calls sub function

10) B (40 8)

Arr[10] has a size of 10*4 40 bytes

When passed to test function array decays to pointers

In 64 bit system pointer size is 8 bytes

11) char 1, int 4, short 2 bytes

Alignment rule requires 4 byte alignment for int, 2 byte alignment for short

So

1 char + 3 padding + 4 int + 2 short + 2 padding = 12 bytes

Eg: struct Test

```
{
    Char a;
    Int b;
}
```

Char - 1 byte, int - 4 byte

Without padding

Address

1000 1001 1002 1003 1004

```
+-----+-----+-----+-----+-----+
| a   | | b   | | b   | | b   | | b   |
+-----+-----+-----+-----+-----+
```

B starts at 1001 but int needs alignment 4

With padding

1000 1001 1002 1003 1004 1005 1006 1007

1001 to 1003 => padding

Union uses shared memory => size of the largest memory

typedef creates a alias, makes it treat the alias as a datatype

13) C (30)

Point p = {10, 20}; initializes p.x = 10 and p.y = 20

Point *ptr = &p; points to p.ptr->x evaluates to 10, and (*ptr).y evaluates to 20.

Sum = 10 + 20 = 30

14) b (200)

h1.ptr = malloc() allocates heap memory

Holder h2 = h1 performs a copy, points to exact same address

Updates value to 200

15) A (Missing free(ptr) before conditional return)

Dynamic memory allocated using malloc()

if(some_condition) exists the function without calling free, causing memory leak

16) B undefined heap corruption

Calling free once releases alloc block back to heap manager's free list

Calling free again without realloc results corrupts internal heap

17) B (causes memory leaks if realloc fails and returns NULL)

Assigning result directly back to the original ptr overwrites the only reference to the original block, causing memory leaks

18) B ([1] followed by [1, 2])

Default args in python are evaluated once at func definition not every time called

First 1 gets added, second call appends 2 so 1, 2

19) B (" and "Pyt")

Unlike direct indexing python slicing handles out of bounds robustly

A[10:15] start beyond length and results in empty string

S[-20:3] clamps neg index to 0 and slices up to index 3 so Pyt

20) B (true False)

Python pre allocates and caches integers from -5 to 256 for optimization

A = 256, b = 256 points to same exact cached obj in memory so true

C = 257, d = 257 exceeds range so false

21) B ([[1, 0, 0], [1, 0, 0], [1, 0, 0]])

[0] * 3 creates a list [0, 0, 0]

[[0] * 3] * 3 create a list all 3 reference to the same inner list obj

Modifying matrix[0][0] = 1 updates first element of the shared inner list

22) B (5 and [])

Gen generates iterators that compute values on fly

sum(gen) iterates through the generator completely so 0 + 1 + 4 = 5

list(gen) generator is done so empty list

23) B ([4, 4, 4])

Inside lambda function late binding happens. Loop finishes, the loop variable i holds its answer 2

Now

All the three lambdas see same i

i
|
v
2

Lambda1 ----\

Lambda2 ----- > use i

Lambda3 ----/

Whenever lambda called it looks up to the current i value

To get [0, 2, 4]

```
funcs = [lambda x, i=i: x * i for i in range(3)]
```

```
print([f(2) for f in funcs])
```

```
Lambda x, i =0 : x * 0
```

```
Lambda x, i =1 : x * 1
```

```
Lambda x, i =2 : x * 2
```

24) C (3)

The try block executes and prepares to return 1.

Before the function exits, any associated finally block is guaranteed to execute.

So finally block executes return 3, overrides the return value of try block

25) B(99, 1)

copy.copy(original) creates a shallow copy , shared inner list reference so when changes happens to the original data it is reflected

Deepcopy recursively copies all the structures completely independent

26) B (bob 90)

zip(names, scores) pairs elements into tuples: ('Alice', 85) at index 0, and ('Bob', 90) at index 1.

enumerate() adds index tracking. When idx == 1, it unpacks name = 'Bob' and score = 90, printing Bob 90.

27) C unboundLocalError

Second count is treated as local variable, so increment without assignment raises error

28) B (12%)

29) B (8 days)

30) B (200)

31) B (\$400)

32) D (3:7)

33) A (3/11)

34) C (262, 080)

35) B (3)

36) A (280)

37) C (76)

38) D (30)

39) A (4 hrs 21 9/11 mins)